

1. Banker's Algorithm

◇ Short Working Explanation

Banker's Algorithm is a method used in Operating Systems to prevent deadlocks. Before allocating resources to a process, the system checks whether enough resources will still remain for other processes to complete safely. If the system stays safe, resources are allocated; otherwise, the request is rejected.

The algorithm mainly uses:

- Allocated resources
- Maximum resources needed
- Available resources

After checking these values, it finds a safe order in which processes can execute.



C++ Program — Banker's Algorithm

```
#include <iostream>
using namespace std;

int main() {

    int process = 5;
    int resource = 3;

    int allocation[5][3] = {
        {0, 1, 0},
        {2, 0, 0},
        {3, 0, 2},
        {2, 1, 1},
        {0, 0, 2}
    };
}
```

```

int maximum[5][3] = {
    {7, 5, 3},
    {3, 2, 2},
    {9, 0, 2},
    {2, 2, 2},
    {4, 3, 3}
};

int available[3] = {3, 3, 2};

int need[5][3];
bool completed[5] = {false};

// Need matrix calculation
for (int i = 0; i < process; i++) {
    for (int j = 0; j < resource; j++) {
        need[i][j] = maximum[i][j] - allocation[i][j];
    }
}

int safeSequence[5];
int count = 0;

while (count < process) {

    bool safe = false;

    for (int i = 0; i < process; i++) {

        if (!completed[i]) {

            int j;

            for (j = 0; j < resource; j++) {

                if (need[i][j] > available[j]) {
                    break;
                }
            }
        }
    }
}

```

```

        if (j == resource) {

            for (int k = 0; k < resource; k++) {
                available[k] += allocation[i][k];
            }

            safeSequence[count] = i;
            count++;
            completed[i] = true;
            safe = true;
        }
    }

    if (!safe) {
        cout << "Unsafe State Detected";
        return 0;
    }

    cout << "System is in Safe State\n";
    cout << "Safe Sequence: ";

    for (int i = 0; i < process; i++) {
        cout << "P" << safeSequence[i] << " ";
    }

    return 0;
}

```

2. Round Robin Scheduling

◇ Short Working Explanation

Round Robin Scheduling is a CPU scheduling technique where each process gets CPU time for a fixed interval called a time slice or quantum. Once the given time expires, the process moves to the end of the ready queue if it has not completed execution.

This method is commonly used because it gives equal chance to all processes and improves response time.

C++ Program — Round Robin

```
#include <iostream>
using namespace std;

int main() {

    int n = 4;

    int burstTime[4] = {12, 7, 5, 9};
    int remainingTime[4];

    for (int i = 0; i < n; i++) {
        remainingTime[i] = burstTime[i];
    }

    int quantum = 3;
    int currentTime = 0;

    bool finished;

    do {

        finished = true;
```

```

    for (int i = 0; i < n; i++) {

        if (remainingTime[i] > 0) {

            finished = false;

            if (remainingTime[i] > quantum) {

                currentTime += quantum;
                remainingTime[i] -= quantum;

            } else {

                currentTime += remainingTime[i];
                remainingTime[i] = 0;
            }
        }
    }

} while (!finished);

cout << "Execution Completed\n";
cout << "Total Time Taken: " << currentTime;

return 0;
}

```